

Sistema de Georreferenciación de rutas de transporte publico

Carlos Javier Rodriguez Manchola
Centro de la industria la empresa y los servicios
Neiva, Colombia
cjrodriguez801@soy.sena.edu.co

Jésus Ariel González Bonilla
Centro de la industria la empresa y los servicios
Neiva, Colombia
<https://orcid.org/0000-0001-6272-8695>

Resumen

Introducción: Se presenta UrbanTracker, un sistema de geolocalización en tiempo real diseñado para optimizar el transporte público urbano mediante el seguimiento continuo de autobuses y la gestión digital de rutas, vehículos y conductores. El proyecto surge ante la falta de información precisa para usuarios y administradores y la necesidad de soluciones asequibles adaptadas a contextos con recursos limitados (p. ej., ciudades intermedias). La validación inicial se realizó en Neiva (Colombia), pero la arquitectura propuesta es transferible a otras ciudades. La motivación se alinea con experiencias previas donde la información en vivo mejora la percepción de puntualidad y la toma de decisiones (“CITY-RUTA”, 2018; “STOPBUS”, 2016).

Métodos: Se desarrolló una plataforma compuesta por: (i) una app móvil para conductores (GPS del smartphone), (ii) una web pública para usuarios y (iii) un panel web administrativo. El backend sigue una arquitectura hexagonal y modular con separación en domain, application e infrastructure: el modelo de dominio concentra las reglas del negocio; los servicios de aplicación orquestan los casos de uso mediante puertos/contratos; y los adaptadores de infraestructura exponen REST, manejan JPA/PostgreSQL, y conectan con mensajería en tiempo real. Los servicios REST están documentados con OpenAPI/Swagger (EasyRest, 2022), y PostgreSQL gestiona el almacenamiento relacional de rutas, vehículos, usuarios y recorridos. Para el tiempo real se usó un flujo en el que el móvil del conductor publica posiciones por MQTT hacia un broker; el backend recibe esos mensajes, los valida y guarda, y luego envía actualizaciones por WebSockets a los clientes web (“SisMo”, 2016). La seguridad se maneja con JWT. En la interfaz, la visualización del movimiento en el mapa se implementó con Mapbox en modo 3D. Para pruebas, se ejecutó un piloto con 1 ruta y 1 vehículo, simulando envíos periódicos de coordenadas cada 5 s desde el móvil y suscribiendo 5–10 clientes web a las actualizaciones en vivo.

Resultados: UrbanTracker permite visualizar en el mapa (Mapbox 3D) la ubicación de los autobuses en servicio con una frecuencia de actualización de 5 segundos, superando un umbral mínimo de 10 s usado como referencia. La latencia medida entre el envío desde el móvil del conductor y la visualización en la web fue <2 segundos bajo condiciones 4G normales. Se cubren las funcionalidades clave: consulta de rutas, vista de vehículos en tiempo real, inicio/cierre de recorrido por conductores y administración de rutas y flotas desde el panel. En la prueba con 5–10 suscriptores concurrentes, la solución mostró buen comportamiento, en línea con reportes previos del uso de MQTT en rastreo vehicular (“SisMo”, 2016).

Discusión: La solución cumple lo esperado en el piloto y confirma que es viable brindar información actualizada a pasajeros y

herramientas operativas a administradores con infraestructura asequible (smartphones + protocolos ligeros). La arquitectura hexagonal ayuda a separar la captura por MQTT y la difusión por WebSockets, manteniendo responsabilidades claras y facilidades para escalar. Se discuten implicaciones de seguridad y privacidad (uso de JWT y manejo responsable de datos de ubicación) y la compatibilidad con arquitecturas híbridas si crece el histórico de eventos (“Aspectos de ingeniería...”, 2022). La experiencia del usuario fue positiva: el mapa 3D facilita entender dónde está el bus y el panel simplifica tareas de gestión.

Conclusiones: UrbanTracker evidencia que es factible mejorar la experiencia del transporte público local mediante geolocalización en tiempo real con un stack accesible. Los resultados sugieren impacto potencial en la reducción de tiempos de espera y en la eficiencia operativa. Trabajos futuros incluyen integrar más fuentes de datos en tiempo real, refinar la interfaz y ampliar las evaluaciones de usabilidad a mayor escala, manteniendo la portabilidad del enfoque a otras ciudades.

Figura 1. Problema y propuesta de solución

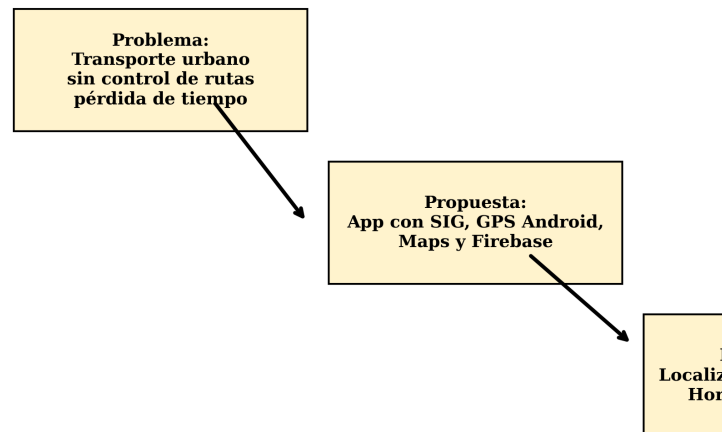


Figura 1: Figura 1. Problema y propuesta de solución

Keywords

geolocalización, transporte público, Spring Boot, React Native, MQTT, WebSockets, JWT

1. Introducción

La falta de información en tiempo real sobre dónde va el bus sigue siendo un problema en muchas ciudades. La gente espera sin saber cuánto falta y los equipos de operación no siempre tienen una vista clara para reaccionar rápido. En otros sectores, como la movilidad y la logística, ya es común usar geolocalización en vivo para mejorar la experiencia y la operación, lo que muestra que estas ideas funcionan cuando se diseñan con eficiencia y pensando en el contexto.

Antes de este proyecto no existía, a nivel local, una plataforma integrada que diera a la vez información en vivo para el pasajero y herramientas de gestión para la operación (rutas, vehículos y conductores) desde el mismo sistema. Con esa necesidad en mente nace UrbanTracker, pensado para ser asequible y replicable en ciudades con recursos limitados.

UrbanTracker es un sistema web+móvil de geolocalización en tiempo real. Incluye una app móvil para conductores (usa el GPS del teléfono), una web pública para las personas usuarias y un panel web para administración. El backend está hecho en Spring Boot y usa una arquitectura hexagonal y modular (carpetas domain, application, infrastructure): el modelo de dominio concentra las reglas del negocio; los servicios de aplicación coordinan los casos de uso por medio de contratos; y los adaptadores exponen REST, se conectan a PostgreSQL/JPA y manejan la comunicación en tiempo real. En la interfaz, el mapa se implementó con Mapbox 3D, que ayuda a entender mejor el movimiento del bus.

Para las actualizaciones en vivo usamos un flujo simple: el móvil del conductor publica posiciones por MQTT hacia un broker; el backend las recibe, las valida y las reenvía por WebSockets a los clientes web [?]. Esto evita depender de consultas periódicas y hace que la actualización se sienta inmediata para el usuario. La literatura en transporte urbano respalda que mostrar la posición en vivo mejora la percepción de puntualidad y la experiencia de viaje [?]; [?].

En el diseño también cuidamos que la API esté bien documentada con OpenAPI/Swagger para integrar sin fricción [?]. PostgreSQL cubre las entidades operativas (rutas, vehículos, conductores, recorridos) y deja abierta la puerta a esquemas híbridos si más adelante se decide guardar históricos de gran tamaño o hacer analítica de trayectorias [?]. En el móvil, elegimos React Native por experiencia del equipo; Flutter se considera una alternativa sólida si más adelante cambian las necesidades [?].

La validación inicial se hizo en Neiva (Colombia) con un piloto de 1 ruta y 1 vehículo. Para probar el tiempo real se enviaron coordenadas cada 5 s desde el teléfono del conductor y se conectaron 5–10 clientes web para recibir las actualizaciones. Estos ensayos muestran que la propuesta es funcional y puede trasladarse a otras ciudades con condiciones similares.

Aportes del trabajo. (i) una arquitectura hexagonal y modular fácil de mantener; (ii) un flujo de tiempo real eficiente basado en MQTT + WebSockets; (iii) una API clara con OpenAPI; (iv) una web pública, un panel y una app de conductor que cubren usuario, operación y conductor en una sola plataforma; y (v) una implementación de bajo costo apoyada en smartphones y protocolos ligeros.

2. Marco teórico y trabajos relacionados

2.1. Transporte público y datos en vivo.

Diversos proyectos muestran que publicar la posición del bus en tiempo real mejora la percepción del servicio y ayuda a planear mejor el viaje. En trabajos previos como CITY-RUTA y STOPBUS se reporta el valor de ofrecer al pasajero rutas y ubicaciones actualizadas en pantalla [?]; [?].

2.2. Mensajería ligera para tiempo real.

En contextos móviles y de IoT, MQTT es muy usado porque los mensajes viajan con poca sobrecarga y se puede ajustar la calidad de servicio (QoS). En rastreo vehicular y seguridad se ha documentado su buen desempeño para enviar posiciones desde teléfonos o dispositivos embebidos, incluso con conectividad limitada [?]. Este tipo de mensajería encaja bien cuando se necesitan actualizaciones frecuentes de ubicación.

2.3. APIs claras y mantenibilidad.

Definir y documentar la API con OpenAPI/Swagger reduce malentendidos y acelera la integración entre equipos y clientes [?]. Además, las publicaciones sobre programación orientada a objetos recomiendan separar responsabilidades, usar contratos claros y evitar dependencias innecesarias para sostener el proyecto en el tiempo [?]. Esto se alinea con una arquitectura hexagonal y modular, donde:

- en la capa de dominio viven las reglas del negocio;
- en la capa de aplicación se agrupan las funciones que resuelven acciones del sistema (por ejemplo, iniciar un recorrido o procesar una posición);
- y en infraestructura están las piezas que hablan con el exterior: los endpoints REST, la conexión con PostgreSQL y la comunicación en tiempo real.

2.4. Datos y crecimiento.

Para este tipo de soluciones, PostgreSQL resulta suficiente para gestionar entidades operativas como rutas, vehículos, conductores y recorridos. Con un buen diseño de tablas e índices se obtiene un rendimiento adecuado sin recurrir a otras familias de bases de datos.

2.5. Marco móvil y mapas.

En el cliente móvil, la elección entre React Native y Flutter depende de la experiencia del equipo y del ecosistema de librerías; ambas opciones son viables para integrar mapas y geolocalización [?]. Para la visualización, herramientas como Mapbox permiten mapas 3D y controles interactivos que mejoran la comprensión del movimiento del bus.

2.6. Síntesis.

Los trabajos revisados respaldan: (i) publicar datos en vivo para mejorar la experiencia del pasajero; (ii) usar MQTT u otros canales ligeros para reducir sobrecarga; (iii) documentar la API y mantener una arquitectura modular/hexagonal para facilitar cambios; y (iv) apoyarse en PostgreSQL para las entidades del sistema. El

vacío común está en guías prácticas que unan todo esto en contextos locales y de bajo presupuesto. UrbanTracker aporta una receta concreta y validaciones iniciales en ese escenario.

3. Metodología

3.1. Proceso de desarrollo

Trabajamos el proyecto por pasos cortos y en iteraciones. Primero definimos qué debía hacer el sistema (a partir del SRS). Después armamos la estructura y repartimos el trabajo por módulos. El orden fue:

- backend con servicios REST y la parte de tiempo real,
- app móvil del conductor,
- web pública para usuarios,
- panel para administración. Usamos GitHub para versionar el código y coordinarnos. En cada vuelta entregamos algo funcional y lo revisamos en equipo.

3.2. Enfoque de validación

Aplicamos una idea simple: planear → hacer → observar → ajustar. Nos enfocamos en que la actualización fuera rápida (alrededor de 5 s), que la latencia móvil→web fuera menor a 2 s, y que el mapa/panel se entendieran sin enredos. La primera validación fue en Neiva (Colombia) con 1 ruta y 1 vehículo.

3.3. Arquitectura y módulos

El backend está organizado en tres partes:

- dominio: donde viven las reglas del negocio (rutas, vehículos, etc.),
- aplicación: donde ponemos las funciones que se encargan de cada acción (por ejemplo, iniciar un recorrido o procesar una ubicación),
- infraestructura: donde conectamos con el mundo exterior: endpoints REST, PostgreSQL y la comunicación en tiempo real.

Para las actualizaciones en vivo usamos un flujo muy directo: el teléfono del conductor envía su ubicación por MQTT a un broker; el backend la recibe, la revisa y la reenvía por WebSockets a la web [?]. La API está documentada con OpenAPI/Swagger para probar e integrar sin enredos [?]. En la web, el mapa está hecho con Mapbox 3D.

3.4. Datos e instrumentos

Durante el desarrollo guardamos issues, pull requests y comentarios del equipo. Además, usamos scripts sencillos para generar rutas de prueba (coordenadas) y así verificar el movimiento en el mapa.

3.5. Pruebas y criterios de aceptación

Funcionales: ingreso del conductor, elegir ruta, iniciar/cerrar recorrido, consultar rutas en la web y usar el panel (crear/editar rutas, vehículos y conductores).

Tiempo real: envíos cada 5 s desde el móvil y recepción en la web; latencia medida <2 s en 4G normal.

Estabilidad: cortes y reconexiones de red; el móvil intenta de nuevo sin que el usuario haga nada.

Figura 2. Arquitectura general de comunicación (UrbanTracker)

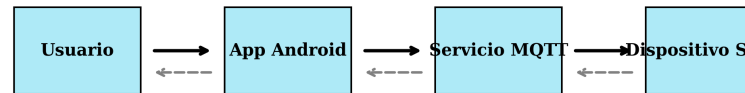


Figura 2: Figura 2. Arquitectura general de comunicación (UrbanTracker)

Criterios de aceptación: actualización ≤ 10 s, latencia <2 s, mapa fluido y 5–10 usuarios conectados a la vez sin problemas.

Estos puntos coinciden con trabajos que recomiendan mensajería ligera para ubicación en vivo y APIs claras para evitar malentendidos [?]; [?]; [?]; [?].

3.6. Alcances y límites

Lo probado fue un piloto (1 ruta/1 vehículo) y 5–10 clientes web. Los resultados dependen de la calidad de la red y de dónde se despliegue el backend.

Las métricas usadas (frecuencia y latencia) reflejan lo que la gente siente al usar la app.

Para cuidar la validez, dejamos criterios claros antes de probar, registramos decisiones y usamos JWT para proteger el acceso.

4. Implementación

4.1. Diseño del sistema

UrbanTracker está dividido en frontend y backend. En el lado de usuario hay tres piezas:

- Web pública para consultar rutas y ver los buses en el mapa.
- App móvil del conductor (React Native) que toma el GPS del teléfono.
- Panel web para administración de rutas, vehículos y conductores.

El backend está hecho en Spring Boot y expone servicios REST. La API está documentada con OpenAPI/Swagger para facilitar pruebas e integración [?].

4.2. Arquitectura del backend (hexagonal y modular)

El backend sigue una arquitectura hexagonal organizada por módulos.

domain: reglas del negocio (rutas, vehículos, conductores, recorridos).

application: funciones que se encargan de cada acción del sistema (por ejemplo, iniciar un recorrido o procesar una ubicación recibida).

infrastructure: adaptadores que publican REST, hablan con PostgreSQL/JPA y gestionan la comunicación en tiempo real.

Esta separación ayuda a mantener el código claro y fácil de entender (POO y separación de responsabilidades) [?].

4.3. Aplicaciones de usuario

Web pública (React/Next.js): muestra rutas y los buses moviéndose en el mapa, y permite filtros básicos.

Panel administrativo (React/Next.js): alta/edición de rutas, vehículos y conductores, con una vista general de la flota.

App del conductor (React Native): pide permisos de ubicación, permite iniciar/cerrar recorrido y envía periódicamente su ubicación.

4.4. Comunicación en tiempo real

El tiempo real funciona con un flujo simple:

El teléfono del conductor envía su posición por MQTT a un broker.

El backend recibe esos mensajes, los revisa y los reenvía por WebSockets a los clientes de la web [?].

Este enfoque reduce esperas para el usuario y evita depender de consultas periódicas. La utilidad de mostrar posiciones en vivo está documentada en trabajos previos [?]; [?].

4.5. Base de datos

Se usa PostgreSQL para las entidades operativas del sistema (rutas, vehículos, conductores, asignaciones/recorridos). No se guardan trazas de posición en esta etapa.

4.6. Seguridad y control de acceso

El acceso a operaciones de conductor y administración se protege con JWT. Los roles limitan lo que cada perfil puede hacer desde la web o el panel.

4.7. Integración de mapas

Para la visualización se integra Mapbox en modo 3D en la web. Esto ayuda a entender mejor el movimiento de cada bus sobre su ruta y mejora la experiencia de quien consulta.

5. Resultados

5.1. 1) Funcionalidades implementadas

Web pública: permite buscar rutas y ver en el mapa (Mapbox 3D) los buses que están en servicio. Cada vehículo aparece como un marcador que se mueve automáticamente a medida que llegan nuevas posiciones [?]; [?].

App del conductor (React Native): el conductor inicia sesión, inicia/cierra recorrido y la app envía su ubicación cada pocos segundos de forma automática.

Panel administrativo (web): creación y edición de rutas, vehículos y conductores, con una vista general para revisar el estado de la flota.

Estas tres partes funcionan juntas y cubren lo esencial: lo que ve la persona usuaria, lo que hace el conductor y lo que gestiona la administración.

5.2. 2) Desempeño en tiempo real

Frecuencia de actualización: alrededor de 5 segundos entre cada posición que envía el teléfono del conductor.

Latencia extremo a extremo: <2 segundos desde que el móvil envía la coordenada hasta que aparece en la web (en 4G normal).

Concurrencia inicial: con 5–10 clientes conectados al mismo tiempo, no se observaron demoras visibles.

Este comportamiento se logra con el flujo simple definido: el móvil publica por MQTT, el backend recibe y reenvía por WebSockets a la web. Esta forma de comunicación ligera coincide con experiencias previas en rastreo con móviles [?] y con la idea de mostrar posiciones en vivo al pasajero [?]; [?].

5.3. 3) Validación del piloto

Alcance: pruebas en Neiva (Colombia) con 1 ruta y 1 vehículo.

Criterios de aceptación verificados: actualización ≤ 10 s (se logró 5 s), latencia <2 s, y estabilidad con 5–10 clientes conectados.

Recuperación ante fallos: si se corta la red móvil, la app del conductor reintenta y vuelve a enviar cuando hay señal.

Contar con una API clara (documentada con OpenAPI/Swagger) ayudó a evitar confusiones en las integraciones y a repetir pruebas de forma rápida [?].

5.4. 4) Lo que quedó fuera (para futuras fases)

No se implementaron funciones como pagos, reservas o predicción avanzada de llegada. La versión actual se concentró en posición en tiempo real y gestión operativa.

5.5. 5) Resumen

UrbanTracker cumple su objetivo: dar visibilidad en vivo del servicio y facilitar la gestión diaria. La experiencia en el mapa es fluida y el panel simplifica tareas clave. Con los resultados del piloto, la solución se ve lista para crecer a más rutas y más vehículos, manteniendo el mismo esquema de comunicación.

6. Discusión

6.1. Qué confirman los resultados.

Lo obtenido muestra que sí es posible mejorar la experiencia del pasajero y dar herramientas útiles a la operación con una plataforma de tiempo real. Ver el bus moverse en el mapa reduce la incertidumbre de espera, tal como señalan trabajos previos en transporte [?]; [?]. Del lado operativo, tener un panel con rutas, vehículos y conductores en un solo lugar facilita decisiones rápidas.

6.2. Ajuste al contexto local.

UrbanTracker apunta a bajo costo y fácil adopción: se usa el teléfono del conductor como fuente de GPS y tecnologías abiertas (MQTT, WebSockets, REST). Esto evita instalar hardware especializado y hace viable la implementación en ciudades con presupuestos limitados. A diferencia de soluciones que se enfocan solo en el pasajero, aquí también se cubre la gestión diaria.

6.3. Elecciones técnicas que funcionaron.

Tiempo real simple: el móvil publica por MQTT, el backend recibe y reenvía por WebSockets. Con esto se logró 5 s de actualización y <2 s de latencia sin complicar la infraestructura [?].

Arquitectura hexagonal y modular: separar dominio, aplicación e infraestructura ayudó a mantener el código claro y a introducir cambios sin “romper” lo ya hecho [?].

API documentada: usar OpenAPI/Swagger redujo malentendidos al integrar web, móvil y backend [?].

Mapa 3D: Mapbox aportó una visual clara del movimiento del bus y mejoró la lectura de la ruta.

6.4. Precisión y uso del teléfono.

El GPS del smartphone dio una precisión suficiente para el objetivo (ubicar el bus en su ruta). El intervalo de 5 s resultó un buen equilibrio entre fluidez y batería; si más adelante se necesita, se pueden hacer ajustes dinámicos (menos envíos cuando el bus está detenido y más cuando está en movimiento).

6.5. Seguridad y privacidad.

Aunque se muestra la ubicación del vehículo (no del conductor como persona), es importante tratar estos datos con cuidado. Se usó JWT para acceso y se limitó la visibilidad: el público ve la localización del bus; los detalles sensibles quedan para perfiles autorizados. Como práctica general, conviene no conservar recorridos más allá de lo necesario y aplicar medidas básicas de protección en los canales.

6.6. Límites del piloto.

Los resultados vienen de un escenario acotado: 1 ruta, 1 vehículo y 5–10 clientes conectados. El desempeño depende de la calidad de la red y de dónde esté el backend. Aun así, lo observado da una base sólida para crecer.

6.7. Qué sigue (oportunidades claras).

Ampliar cobertura: más rutas y más vehículos para medir con mayor variedad de escenarios.

Métricas continuas: instrumentar mediciones (frecuencia, latencia, reconexiones) para ver tendencias.

Funciones para el usuario: estimación de tiempos de llegada y alertas.

Operación: monitoreo más fino de eventos y salud del sistema.

Móvil: seguir con React Native y, si la escala lo amerita, compilar con Flutter en un módulo puntual [?].

6.8. Cierre.

En conjunto, UrbanTracker se alinea con lo que recomiendan las publicaciones sobre transporte y software práctico: datos en vivo para el pasajero [?]; [?], mensajería ligera para mover posiciones [?] y APIs claras con una arquitectura modular que facilite el mantenimiento [?]; [?]. Los resultados del piloto son un buen punto de partida para escalar y agregar funciones sin perder la sencillez de la solución.

7. Conclusiones

Se desarrolló e implementó UrbanTracker, una plataforma de geolocalización en tiempo real que integra lo que necesitan usuarios, conductores y administradores en un solo lugar. Con un backend en Spring Boot (arquitectura hexagonal y modular), web en React/Next.js, app de conductor en React Native y mapas Mapbox 3D, la solución cumple su objetivo principal: ver el bus en vivo y gestionar la operación diaria sin depender de hardware costoso.

En las pruebas del piloto (Neiva, 1 ruta/1 vehículo) se alcanzaron actualizaciones 5 s y latencia <2 s del teléfono del conductor a la web, cumpliendo con los criterios de aceptación. Esto es coherente con la idea de que mostrar la posición en tiempo real mejora la experiencia del pasajero [?]; [?]. El uso de WebSockets/MQTT permitió mantener la comunicación fluida, y contar con una API clara documentada con OpenAPI/Swagger simplificó la integración entre componentes [?]. Para los datos operativos (rutas, vehículos, conductores y recorridos), PostgreSQL resultó adecuado, en línea con recomendaciones de diseño y mantenibilidad [?].

En términos de impacto, UrbanTracker ayuda a reducir la incertidumbre de espera en los usuarios y brinda a los administradores una vista centralizada para tomar decisiones rápidas. Además, al aprovechar smartphones y protocolos ligeros, baja la barrera de entrada y favorece la adopción en contextos con presupuesto limitado.

7.1. Trabajo futuro.

Funciones para el usuario: estimación de tiempos de llegada y alertas.

Operación y datos: métricas continuas y, si se requiere, almacenar históricos de recorridos para análisis y planificación.

Seguridad y privacidad: revisar políticas de retención y controles de acceso más finos con JWT.

Escala y usabilidad: ampliar a más rutas/vehículos y realizar pruebas de usabilidad con más personas.

En síntesis, UrbanTracker demuestra que es viable y útil implementar una plataforma de seguimiento en tiempo real para el transporte público con tecnologías accesibles, manteniendo una base sólida para crecer y sumar capacidades [?]; [?]; [?]; [?].

A. Checklist de reproducibilidad (UrbanTracker)

■ Datos:

- Piloto en Neiva (Colombia) con 1 ruta y 1 vehículo.
- Posiciones GPS emitidas desde el **smartphone del conductor** cada ~5 s.
- En esta fase **no se guardan** trazas de posición.

■ Código:

- Repositorio: <https://github.com/AFSB114/UrbanTracker.git>
- Rama: dev
- Ejecución: **levantar el archivo YML de la raíz con Docker** para subir todos los servicios.
- Variables de entorno usadas:
 - NEXT_PUBLIC_MAPBOX_TOKEN = pk.eyJ1IjoieWZzYjExNCIsImEiOiJjbWl1bmN2OGYxanloMmlvbjd0dndtb3g5In0.2ON4hP04tvToiU_p_IsHbg
 - NEXT_PUBLIC_API_URL = <http://backend:8080>

■ **Entorno:**

- Java (JDK): **17**
- Node.js: **v22.17.0**
- PostgreSQL: **16**
- Mosquitto: **imagen de Docker**

■ **Procedimiento:**

1. Definir las variables de entorno indicadas.
2. Levantar los servicios con Docker usando el **YML de la raíz**.
3. Ver la web y el **mapa (Mapbox 3D)**.
4. Probar envío de ubicaciones desde el móvil del conductor cada ~5 s.
5. Conectar **5–10** clientes web y verificar la actualización en vivo.

■ **Resultados:**

- Frecuencia de actualización: ~5 s.
- Latencia extremo a extremo (móvil → web): < 2 s (4G).
- Concurrencia validada: **5–10** clientes conectados sin demoras visibles.

Referencias

2016. SisMo: Sistema de seguridad para motocicletas. https://web.archive.org/web/20180415091800id_/http://revistas.usc.edu.co/index.php/Ingenium/article/viewFile/651/518
2022. Aspectos de ingeniería de software, bases de datos relacionales y no relacionales y como servicios en la nube para el desarrollo de sistemas de software híbridos. https://sedici.unlp.edu.ar/bitstream/handle/10915/144325/Documento_completo.pdf-PDFA.pdf?sequence=1
2022. Desarrollo híbrido con Flutter. <https://ciencialatina.org/index.php/cienciala/article/view/2959/4350>
2022. EasyRest: Generador automático de API REST basado en Spring Framework y motor de plantillas Beetl. <https://d1wqtxs1xzle7.cloudfront.net/111860414/18841-libre.pdf>
2023. El papel de la programación orientada a objetos en el desarrollo de software sostenible y escalable. <https://ve.scielo.org/pdf/uct/v27n121/2542-3401-uct-27-121-85.pdf>